Propuesta de desarrollo colaborativo y automatización con un sistema de incidencias en C#

Estudiante 1: _____

Estudiante 2: _____

Institución: _____

Asignatura: Programación Avanzada en C#

Curso y paralelo: _____

Docente: _____

Fecha de entrega: _____

Introducción

La empresa de desarrollo de software descrita en el caso presenta pérdida de versiones, poca organización del equipo, errores en producción y documentación insuficiente. Estos problemas se relacionan entre sí. Cuando un cambio no tiene una versión identificable, resulta difícil conocer qué se entregó, quién debe revisarlo o cómo volver a una versión estable. Una herramienta aislada no corrige ese proceso.

Se propone un modelo de trabajo con Git y GitHub, especificaciones verificables, revisión de cambios y automatización con GitHub Actions. GitHub Copilot puede apoyar tareas acotadas, mientras que el estudiante conserva la responsabilidad de comprender y validar las sugerencias. La propuesta se adapta a un equipo de dos estudiantes y a una implementación gradual en una empresa pequeña de Ecuador.

El ejemplo práctico es una aplicación Windows Forms para gestionar incidencias. Su propósito es demostrar cómo un requisito llega a una función comprobada y a un paquete distribuible. El alcance comprende registro, modificación, eliminación, atención, resolución, reapertura y consultas. Se usa persistencia XML local y una solución con carpetas Entidades, Controlador y Formularios. La aplicación permite explicar el proceso completo sin introducir un servidor ni numerosas pantallas.

Desarrollo de los temas

Control de versiones y Git avanzado

Git conserva una historia de cambios mediante commits. Las ramas permiten desarrollar una modificación sin alterar inmediatamente la versión principal. En este proyecto, cada commit debe describir una intención pequeña y comprobable. Antes de confirmar, git diff permite revisar el contenido; git log --graph ayuda a observar cómo se relacionan las ramas. El libro oficial explica que el uso de ramas debe responder al flujo del equipo (Chacon y Straub, s. f.).

Las operaciones avanzadas se aplican a problemas concretos. git revert agrega un commit que revierte una modificación integrada y conserva la historia. git cherry-pick permite llevar una corrección específica a otra rama. git bisect busca el commit que introdujo una regresión cuando existe una comprobación reproducible. Un rebase reorganiza commits de una rama, pero exige coordinación si otros ya trabajan sobre esa historia. La propuesta evita reescribir main y reserva esas operaciones para necesidades justificadas.

Git Flow y Trunk Based Development

Git Flow separa ramas de desarrollo, funcionalidades, versiones y correcciones urgentes. Puede servir cuando una empresa mantiene varias versiones publicadas. Su costo es coordinar más ramas e integraciones. Driessen (2010) describió el modelo y posteriormente aclaró que un flujo más sencillo puede ajustarse mejor a entregas continuas. Por eso no se adopta Git Flow únicamente porque tenga más pasos.

Trunk Based Development busca integrar cambios pequeños y frecuentes en una línea principal. Las ramas deben durar poco y las pruebas deben ofrecer respuesta rápida. DORA (s. f.-b) relaciona esta práctica con la reducción de integraciones extensas. Para dos estudiantes se propone GitHub Flow, con main y ramas cortas revisadas mediante pull requests. La integración diaria es una meta del equipo. Usar una rama feature no demuestra por sí solo que se esté aplicando Trunk Based.

GitHub y revisión colaborativa

GitHub aloja el repositorio y relaciona el código con discusiones, incidencias y revisiones. Un pull request presenta una propuesta de integración entre ramas. Debe explicar qué problema resuelve, qué comportamiento cambia y cómo se verificó. En GitHub Flow, la rama permite preparar el cambio y solicitar observaciones antes de incorporarlo a la rama principal (GitHub, s. f.-d).

La revisión debe comprobar reglas y resultados. En el ejemplo de reapertura, el revisor verifica que solo una incidencia resuelta vuelva a pendiente y que se elimine la fecha de cierre.

También comprueba los casos rechazados, la conservación de datos y la claridad del código. El otro estudiante debe dejar una revisión real. Un PR creado con una sola cuenta o una comprobación automática no demuestra participación de dos personas.

Desarrollo basado en especificaciones y Spec Kit

Spec Driven Development establece el comportamiento esperado antes de implementar. Una especificación útil define quién necesita una función, cuál es su resultado y cómo reconocer que funciona. En HU02, CA08 exige que reabrir una incidencia resuelta cambie su estado a Pendiente y borre la fecha de cierre. CA09 exige que los intentos inválidos conserven los datos. Estos criterios permiten revisar el programa sin depender de una descripción ambigua.

GitHub Spec Kit proporciona un proceso y herramientas para trabajar con principios, especificación, aclaraciones, planificación y tareas (GitHub, s. f.-e). El repositorio contiene constitution.md, spec.md, plan.md y tasks.md redactados con ese enfoque. Esta estructura no certifica una ejecución del CLI. La guía del repositorio indica cómo inicializar la herramienta oficial con la integración de Copilot en una rama de prueba y revisar los archivos generados. Se distingue así el uso de SDD de una sesión todavía no realizada con Spec Kit.

Aplicación de las seis fases de SDD

La aclaración del docente Joffre establece que primero se investiga SDD y después se aplica el caso práctico con código generado por IA desde los requerimientos. Los estudiantes precisan el comportamiento, revisan la especificación y comprueban los resultados. Por ello, la evidencia debe relacionar las reglas con el código generado y sus verificaciones, no limitarse a copiar una respuesta de la IA.

En la primera fase, Requerimientos, se identifican seis necesidades R01 a R06 en docs/requisitos.md. Abarcan administración de incidencias, estados, consultas, conservación de datos, estructura sencilla y entrega verificable. En la segunda fase, Especificación, HU01 a HU03 concretan esas necesidades en CA01 a CA16, con entradas, resultados y restricciones.

En la tercera fase, Plan técnico, plan.md distribuye responsabilidades entre entidad, lista, archivo XML, formularios y pruebas; tasks.md organiza el trabajo. En la cuarta fase, Implementación, Codex genera y ajusta el código conforme a esas reglas. En esta ampliación se conservan las pantallas y se agregan comprobaciones al proyecto Pruebas.

En la quinta fase, Verificación, se ejecutan las reglas y los eventos reales de los formularios. La matriz docs/trazabilidad.md relaciona los 16 criterios con su implementación y evidencia. En la sexta fase, Evolución, los cambios se registran en ramas, commits y PR; se actualiza la especificación y se repiten las verificaciones antes de distribuir una versión.

El historial no documenta cada mensaje de generación ni prueba una aprobación independiente de todos los requisitos antes del código original. El ejercicio distingue ese límite de las evidencias comprobables. El Issue número 5 y el PR número 6 reúnen esta ampliación. La revisión humana del segundo estudiante sigue pendiente en el Issue número 3.

DevOps y GitHub Actions

DevOps acerca desarrollo y operación mediante responsabilidades compartidas y retroalimentación sobre la entrega. La integración continua ejecuta comprobaciones frecuentes para detectar cambios incompatibles. DORA (s. f.-a) describe la importancia de integrar lotes pequeños y mantener pruebas rápidas. El equipo necesita atender un fallo del proceso antes de seguir acumulando cambios.

El workflow del ejemplo obtiene el código, prepara MSBuild, compila con advertencias tratadas como errores y ejecuta las pruebas. Si alguna comprobación falla, su código de salida detiene la generación del paquete. Cuando todo pasa, Actions conserva un ZIP. Una etiqueta de versión v* activa además la distribución del paquete mediante una release privada. La documentación de GitHub explica la automatización de compilación y pruebas de .NET (GitHub, s. f.-c).

En esta aplicación de escritorio, entregar el ejecutable validado constituye la demostración de distribución. No representa una instalación automática en los equipos de una

empresa. Antes de una adopción real se necesitan un instalador, firma del ejecutable, respaldo, controles de acceso y seguimiento de errores. El pipeline reduce tareas repetitivas, pero depende de que sus comprobaciones representen riesgos relevantes.

GitHub Copilot y elaboración de prompts

GitHub Copilot puede proponer fragmentos, explicar código y ayudar a preparar pruebas. Sus sugerencias requieren revisión porque pueden ser incompletas o equivocadas. La guía de uso responsable recomienda valorar sus límites y comprobar el resultado (GitHub, s. f.-b). En el proyecto se prepararon instrucciones de estilo para conservar campos privados, constructores explícitos y llaves en todos los bloques.

Un prompt útil proporciona contexto, objetivo, restricciones y verificación. Ejemplo: «Lee CA08 y CA09. Propón pruebas de reapertura para los estados Resuelta, Pendiente y En atención. Mantén las carpetas actuales y las propiedades explícitas. No agregues formularios. Ejecuta las pruebas y explica los resultados». El estudiante debe comparar la respuesta con la especificación. La preparación se hizo con asistencia de Codex. No se presenta como evidencia una sesión de GitHub Copilot que todavía no haya ejecutado el grupo.

TDD y calidad del software

Test Driven Development propone escribir una prueba que revele una regla pendiente, implementar lo necesario para que pase y mejorar el código conservando el resultado. La prueba debe fallar por el comportamiento esperado y no por una configuración rota. Microsoft recomienda que las pruebas sean claras y comprueben comportamientos relevantes para evitar regresiones (Reese, s. f.).

La demostración de reapertura conserva un commit con una regla sin implementar y tres comprobaciones fallidas. El siguiente commit implementa la transición y deja las 29 comprobaciones aprobadas en la ejecución local. Las pruebas cubren datos obligatorios, estados, fechas, CRUD, recuperación de XML y archivos inválidos. Se usa una consola propia que devuelve un código distinto de cero ante un fallo. No se denomina MSTest a este ejecutor.

Las pruebas de lógica se complementan con revisión visual y una demostración manual de la interfaz.

Verificación ampliada de los criterios

La suite actual conserva las 29 comprobaciones originales y agrega 21 de integración con formularios: 50 en total. Se prueban los filtros sin restricciones, por estado, por nombre parcial, combinados y sin coincidencias. También se comprueba que el resumen agrupe todos los registros aunque haya filtros activos.

Los casos de administración ejecutan Nuevo y Modificar, verifican el código automático y la conservación de identidad y fechas, guardan, cancelan y responden a la confirmación de eliminación. Los casos de error impiden escribir en una ruta de prueba para verificar que agregar, modificar, cambiar estado o eliminar no alteren la lista ni el archivo anterior. Un XML dañado bloquea las seis acciones de edición.

Las pruebas usan archivos aislados junto al ejecutor, sin leer el XML personal. Los diálogos se contestan dentro del hilo de pruebas, sin usar el teclado global ni otras aplicaciones. Actions comprueba además CA14 mediante el analizador de sintaxis C# de Visual Studio y compila tratando advertencias como errores. Un resultado verde corresponde a los escenarios configurados y no implica cobertura total de todas las entradas posibles ni sustituye la revisión del compañero.

Issues Projects y Milestones

Un Issue describe una necesidad, un defecto o una tarea con criterios de aceptación. Un Milestone agrupa entregables hacia una versión o fecha. GitHub Projects permite organizar Issues y pull requests en una tabla, tablero o cronograma (GitHub, s. f.-a). Su valor depende de actualizar el estado y de mantener una relación con el trabajo real.

Se propone un tablero con Pendiente, En progreso, En revisión y Finalizado. Cada tarea debe tener un responsable y enlazar la especificación o el PR. Para este ejercicio, la reapertura tiene un Issue asociado al PR número 1. La planificación considera una primera semana para

especificación y prototipo y una segunda para pruebas, revisión y exposición. Los roles y fechas indicados en la propuesta son una organización sugerida, no un registro inventado de participación del grupo.

Análisis crítico

La pérdida de versiones no se resuelve enviando copias con nombres como final2 o definitivo. El repositorio debe ser la referencia del equipo, con commits identificables y entregas etiquetadas. Sin embargo, Git tampoco reemplaza el respaldo de datos ni las decisiones sobre acceso. El beneficio depende de que los integrantes confirmen cambios con regularidad y sepan recuperar una versión.

La automatización ofrece respuesta consistente, pero un pipeline verde solo demuestra que pasaron las comprobaciones configuradas. Si una prueba omite una regla importante, esa regla puede fallar en uso real. Por ello se relacionan criterios de aceptación y pruebas, se revisa el formulario y se proponen escenarios manuales. Las validaciones del XML protegen el prototipo de archivos inválidos, pero no aportan la concurrencia ni la auditoría necesarias para producción.

La IA reduce el esfuerzo de escribir una primera propuesta, aunque puede introducir lógica difícil de explicar. El grupo necesita revisar los cambios y justificar cada herramienta. Añadir formularios, bibliotecas o servicios sin necesidad aumentaría la dificultad del ejercicio. El modelo propuesto favorece una pantalla principal con pestañas y reglas simples. La revisión humana permanece necesaria para valorar el comportamiento y la legibilidad.

También existe un límite académico: configurar herramientas desde una cuenta no reemplaza la colaboración de dos estudiantes. Cada integrante debe comprender una parte del sistema, revisar la parte del compañero y participar en la exposición. El informe conserva espacios para los datos del grupo y diferencia la evidencia automatizada de las actividades que los estudiantes deben realizar personalmente.

Propuesta de solución para la empresa

Implementación gradual

La primera etapa consiste en identificar un producto piloto, crear un repositorio privado y acordar una especificación breve. El responsable del producto prioriza los problemas y el equipo define criterios verificables. La segunda etapa introduce ramas cortas, PR y revisión entre compañeros. La tercera automatiza compilación, pruebas y distribución. La cuarta incorpora Copilot con reglas de revisión y evalúa Spec Kit para funciones que necesiten aclaración o trazabilidad adicional.

El equipo debe conservar un registro de fallos de producción y relacionarlos con una prueba cuando sean reproducibles. Antes de incorporar una función, el revisor comprueba criterios, calidad y resultado del pipeline. Una versión aprobada recibe una etiqueta. Si surge una regresión, el equipo puede recuperar la versión anterior y preparar una corrección identificable. La empresa ajustará esta política según su producto y sus obligaciones reales.

Organización de las dos semanas

Durante la primera semana, el estudiante 1 puede preparar los requisitos y la entidad, y el estudiante 2 puede preparar la interfaz y revisar esos requisitos. Después intercambian la revisión. Durante la segunda semana, uno prepara las pruebas y el workflow y el otro valida la demostración, el informe y la presentación. Ambos deben revisar el PR y ensayar el recorrido completo. La distribución propuesta evita que una sola persona concentre todas las decisiones.

Se propone medir el porcentaje de cambios con PR, el tiempo de respuesta del pipeline, los defectos detectados antes de entregar y el tiempo de recuperación ante una regresión. No se asignan porcentajes de mejora sin una línea base. La empresa primero registra una semana de operación y después compara resultados equivalentes. Para el taller se comprueban resultados concretos de compilación y pruebas, sin extrapolarlos a una mejora empresarial medida.

Demostración de la aplicación

En frmPrincipal, el usuario pulsa Nuevo y abre frmAdmin. Allí escribe el título y la descripción, asigna un responsable y selecciona la prioridad. Guardar asigna el código automáticamente. Modificar abre frmAdmin con el registro seleccionado. Cancelar permite salir sin aplicar cambios. Eliminar solicita confirmación. Iniciar atención y Marcar resuelta aplican las reglas de la entidad. Reabrir devuelve una resuelta a pendiente.

En Consultas se puede seleccionar el estado y escribir parte del nombre del responsable. Consultar combina ambos filtros. Resumen por responsable agrupa todos los registros y cuenta los estados mediante LINQ. Los datos permanecen después de cerrar el programa. La ubicación del archivo se documenta para distinguir los datos del usuario del código versionado.

Reflexión del grupo

El problema que resuelve Git

Git permite identificar cambios y recuperar versiones. En este caso reemplaza las copias manuales sin identificación. El equipo necesita commits pequeños que expliquen una intención concreta.

La importancia actual de DevOps

DevOps coordina la entrega y permite detectar fallos antes de publicar. Las pruebas automáticas reducen olvidos, mientras la empresa conserva la responsabilidad de responder a los problemas.

Los cambios que introduce la IA

La IA facilita propuestas y explicaciones. El programador necesita describir reglas y verificar el resultado. Comprender el código sigue siendo necesario para aprender y mantener el sistema.

La herramienta más útil para el caso

GitHub resulta especialmente útil para este caso porque reúne historia y discusión de cambios. Actions agrega una comprobación repetible. La elección responde a las necesidades de la empresa.

Conclusiones

La propuesta relaciona versiones identificables, especificaciones, revisión y pruebas con los problemas de la empresa. Actions automatiza la entrega y la IA apoya tareas revisables. El prototipo permite explicar ese proceso con reglas sencillas y una pantalla principal. Su almacenamiento local limita el uso corporativo. La adopción empresarial requiere validar el proceso, ampliar la infraestructura y mantener la participación del equipo.

Referencias

Chacon, S., y Straub, B. (s. f.). *Pro Git: Branching workflows.* https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows

DORA. (s. f.-a). *Continuous integration.* https://dora.dev/capabilities/continuous-integration/

DORA. (s. f.-b). *Trunk-based development.* https://dora.dev/capabilities/trunk-based-development/

Driessen, V. (2010, 5 de enero). *A successful Git branching model.* https://nvie.com/posts/a-successful-git-branching-model/

GitHub. (s. f.-a). *About Projects.* https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects

GitHub. (s. f.-b). *Application card: GitHub Copilot inline suggestions.* https://docs.github.com/en/copilot/responsible-use/inline-suggestions

GitHub. (s. f.-c). *Building and testing .NET.* https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-net

GitHub. (s. f.-d). *GitHub flow.* https://docs.github.com/en/get-started/using-github/github-flow

GitHub. (s. f.-e). *GitHub Spec Kit [Software].* https://github.com/github/spec-kit

Reese, J. (s. f.). *Unit testing best practices for .NET. Microsoft Learn.* https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices

Apéndice A Evidencias y recorrido práctico

Repositorio privado: https://github.com/Nicolas65a/incidencias-devops-csharp

La reapertura se documenta en el PR número 1 y el Issue número 2. El commit rojo 1c84d6a compila, pero rechaza tres comprobaciones; el verde 7a38862 implementa la regla y aprueba las 29 comprobaciones originales.

Evidencia roja: https://github.com/Nicolas65a/incidencias-devops-csharp/actions/runs/39645265303

Evidencia verde: https://github.com/Nicolas65a/incidencias-devops-csharp/actions/runs/39645598929

La ampliación relaciona los 16 criterios y añade 21 comprobaciones: 50 en total. Su código, documentos y validación se reúnen en https://github.com/Nicolas65a/incidencias-devops-csharp/pull/6

La versión de esta ampliación es v1.1.0. Su publicación y el resultado real de Actions se registran en EvidenciasGitHub.txt. El paquete incluye el ejecutable y los resultados de pruebas y estilo.

Para exponer: registrar una incidencia, rechazar su cierre directo, atender, resolver y reabrir; después consultar y abrir la matriz, el PR y Actions.

El docente necesita acceso al repositorio privado. El grupo debe completar sus nombres, registrar la revisión del segundo integrante y ensayar. La documentación de Copilot y Spec Kit no certifica una ejecución de esas herramientas.

Apéndice B Matriz de criterios y evidencias

La matriz completa del repositorio identifica los métodos y nombres de comprobaciones. Esta síntesis diferencia las pruebas del programa, las de interfaz y la verificación del pipeline. La revisión del segundo estudiante sigue pendiente.

Criterio	Comportamiento	Evidencia
CA01	Código único y estado inicial	Lógica y Nuevo desde el botón
CA02	Campos y prioridad válidos	Lógica y rechazo en el editor
CA03	Modificar y eliminar	Fechas conservadas y confirmación
CA04	Persistencia	XML y reapertura del formulario
CA05	Atender solo pendiente	Transiciones válidas y rechazadas
CA06	Resolver solo en atención	Transiciones válidas y rechazadas
CA07	Fecha de cierre válida	Lógica de fechas
CA08	Reapertura de resueltas	Tres casos y evidencia TDD
CA09	Rechazo sin cambios	Estado y fechas posteriores
CA10	Filtros de consulta	Cinco escenarios de interfaz
CA11	Resumen por responsable	Conteos con filtros activos
CA12	Guardar sin pérdidas	Cuatro operaciones con ruta bloqueada
CA13	Carga dañada	Bloqueo y archivo conservado
CA14	Bloques con llaves	Análisis de sintaxis con Roslyn
CA15	Dos formularios	Guardar, cancelar, Nuevo y Modificar
CA16	Entrega condicionada	Actions y paquete validado